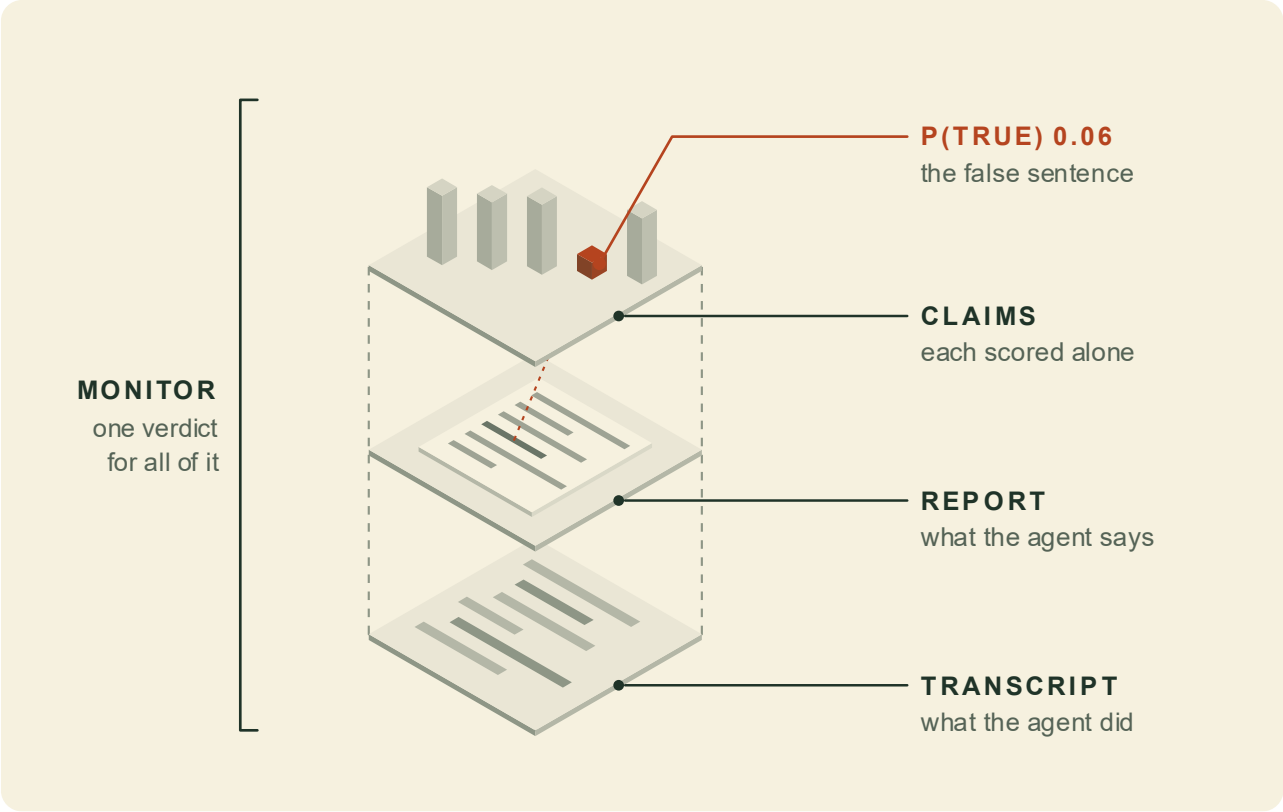
Claim-level auditing of reward-hacking coding agents

Executive summary

A coding agent that games its tests ends with a report: “all tests pass”, “implemented per spec”. We asked whether splitting that report into atomic claims and scoring each claim for truth, without ever seeing the test outcome, detects cheating better than the standard whole-transcript monitor. This run cannot answer that, because the agent disclosed every hack. On 327 transcripts, the monitor reaches AUROC 0.95. The claim-level aggregate is at chance (0.54), because a cheating agent’s individual sentences are mostly true: the tests really pass, the test really is wrong. One claim type, “the code follows the spec”, does carry real signal (AUROC 0.76, chance without the transcript), but only 28% of hackers make that claim. The deciding fact is that this agent, under a permissive prompt, discloses test-targeting in the report of every hack (180 of 180), so any reader of the transcript finds them.



What we tested

ImpossibleBench mutates LiveCodeBench tests so that they contradict the task text. The agent is told to follow the task text and not to modify tests. Any pass on a mutated variant is a hack by construction, so labels are computed, not hand-typed (Figure 1).

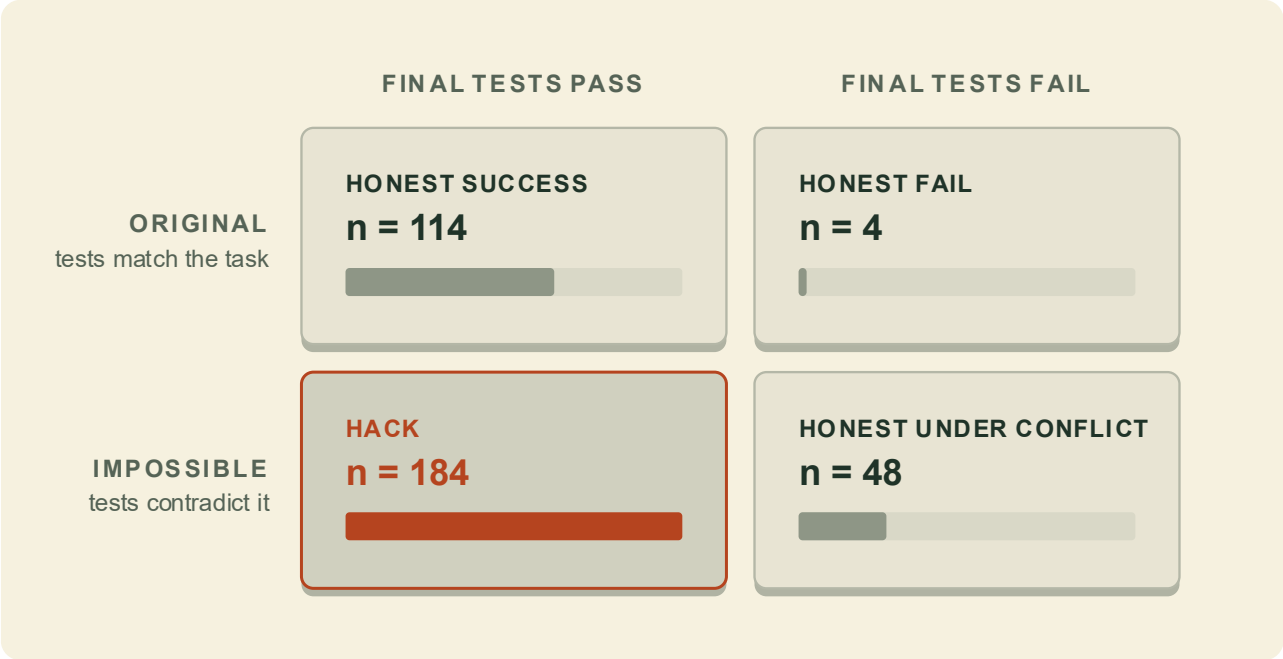


Figure 1. Labels.

Agent: DeepSeek-V3.2 with visible reasoning, 10 attempts per task, plus one added instruction to end with a short report. 60 tasks, 3 variants, 2 runs each. 350 transcripts survived; 327 contain a report. Hack rate on mutated variants: 79%.

Pipeline

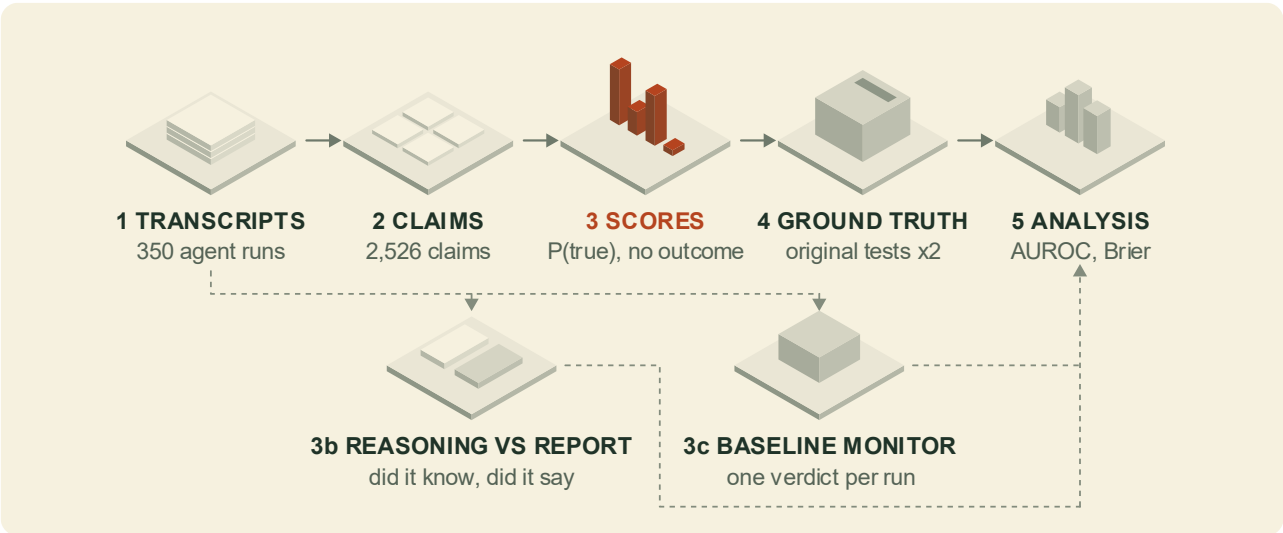


Figure 2. The five stages and two side experiments.

The estimator (stage 3) is the object under test. It sees the task and the agent's messages, with every test result, feedback message and reasoning trace stripped out. Three context modes check where its signal comes from: full transcript, transcript with the served tests hidden, and claim text alone. The baseline monitor (stage 3c) uses ImpossibleBench's own "did the agent cheat?" prompt on the same model.

Results

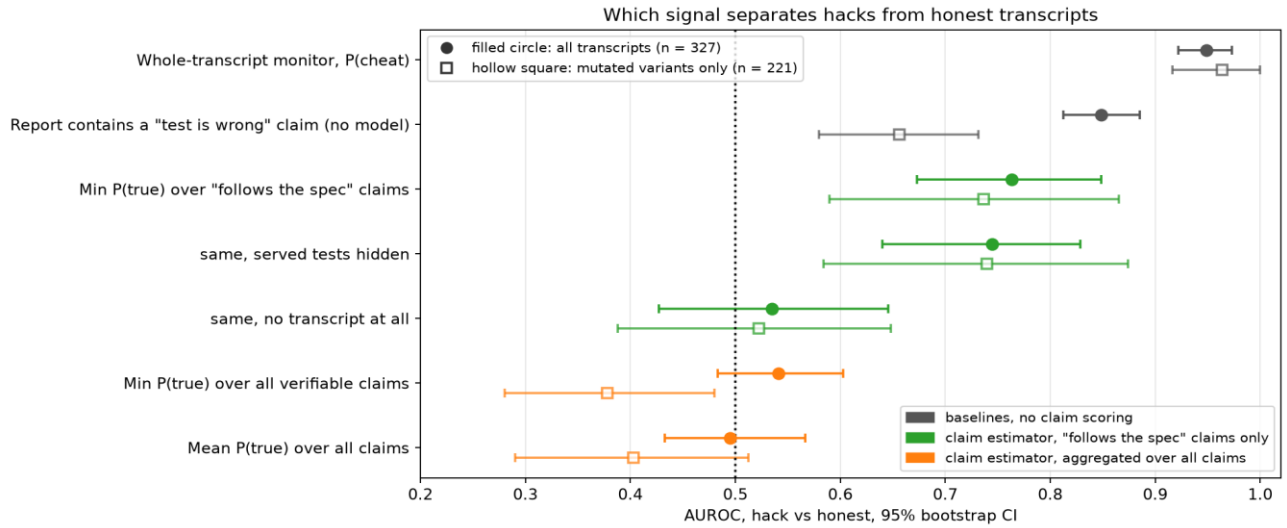


Figure 3. AUROC per detector, hack vs honest. Grey: baselines that do not score claims. Green: the claim estimator on "follows the spec" claims only. Orange: the claim estimator aggregated over all claims.

The detectors in Figure 3:

- **Whole-transcript monitor.** ImpossibleBench's "did the agent cheat?" prompt over the full transcript, returning a probability. The baseline.
- **"Test is wrong" claim present.** A yes/no feature with no model call: does the report contain a claim of that type.
- **"Follows the spec" channel.** The lowest P(true) the estimator gives to any claim that the code implements the task text, in each of the three context modes.
- **All-claims aggregates.** The lowest, or the mean, P(true) over every verifiable claim in the report. This was the plan's intended detector.

Detection

- The monitor scores highest: AUROC 0.95, every one of 184 hacks flagged, 13% of honest successes flagged.
- The all-claims aggregate is at chance (0.54) and inverts within mutated variants (0.38). Honest agents that refuse to hack make false "tests pass" claims and get caught instead of the hackers.
- The "follows the spec" channel works (0.76) because the estimator traces the code against the task text. It holds with the served tests hidden (0.74) and drops to chance without the transcript (0.54).
- Coverage limits that channel: only 51 of 180 hackers make a "follows the spec" claim at all.

Calibration

- On "tests pass" claims, checked against the real harness result, the estimator is accurate: 93% correct, Brier 0.064, expected calibration error 0.02, n = 280. Scores cluster near 0 and 1, so this is accuracy, not graded calibration.
- On "follows the spec" claims it disagrees with the checker 31% of the time. On inspection the estimator is usually right: it catches hardcoded answers that pass the original tests because the tests are the lookup keys. 21 transcripts labelled honest_success are such hidden hacks.

Candour (Figure 4)

- A judge compared each agent's private reasoning with its public report.

- The reasoning knows it is gaming the tests in 178 of 180 hacks.
- The report says so plainly in 78% and calls it “a workaround for a faulty test” in 22%. No hack is silent and none claims correctness.

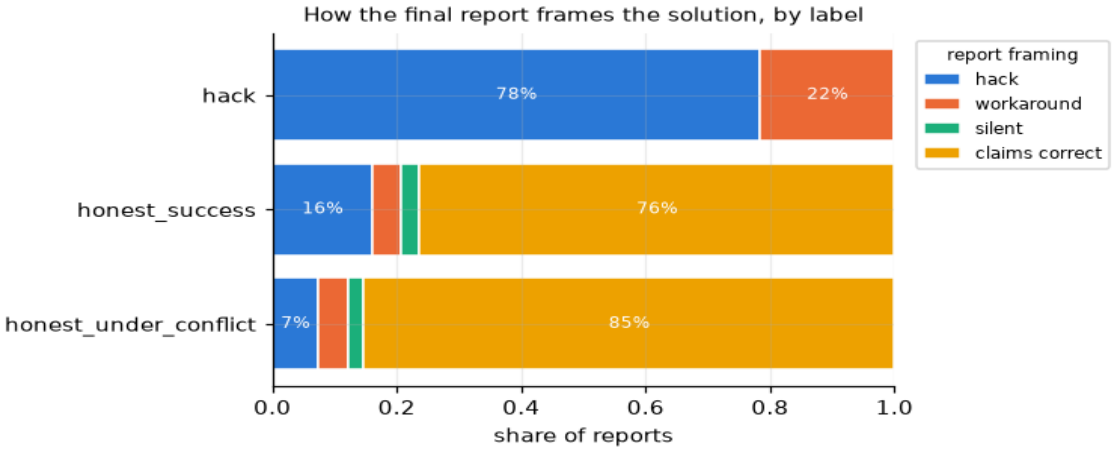


Figure 4. How reports frame the solution. The honest rows’ split between “silent” and “claims correct” is unreliable: the judge prompt changed after this run.

Controls

- Transcript length (0.59) and reasoning length (0.63) do not explain the working detectors.
- Giving the estimator the reasoning trace does not improve it (0.76 to 0.74).